



برنامه نویسی به زبان سی

اشاره گرها

دکتر مهران صفایانی

safayani@iut.ac.ir

safayani.iut.ac.ir



<https://www.aparat.com/mehran.safayani>



https://github.com/safayani/Programming_Basics_course



What is a Pointer?

- **Definition:** A variable whose value is a **memory address**.
- It “points to” another variable where the actual value is stored.
- **Direct vs. Indirect Reference:**
 - A regular variable **directly** references a value (e.g., `int count = 10;`).
 - A pointer **indirectly** references a value (it holds the *address* of count).
 - This process is called **indirection**.

```
[Pointer Variable] ----> [Another Variable's Memory Address] ----> [Actual Value]
```

Why Use Pointers?

- Pointers are essential for advanced C programming. They enable you to:
-  **Pass-by-reference:** Modify the original arguments of a function.
-  **Manipulate arrays and strings** efficiently.
-  **Pass functions** as arguments to other functions.
-  **Create dynamic data structures:**
 - Linked Lists, Stacks, Queues, Trees.
 - Structures that can grow and shrink during program execution

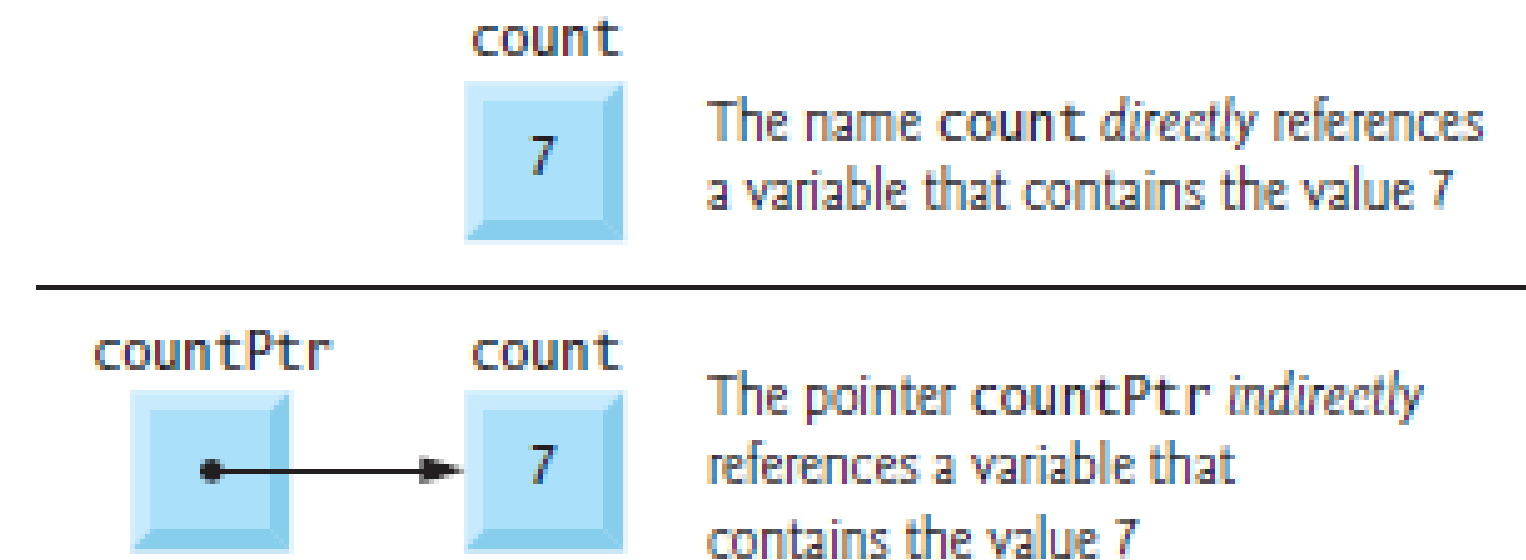
Declaration and Naming

- Syntax: Pointers must be declared before use. The * indicates it's a pointer.

```
// Read right-to-left: "countPtr is a pointer to int"  
int *countPtr;
```

- **Naming Conventions:** It's good practice to make pointer variables obvious.

- End with Ptr (e.g., countPtr)
- Start with p or p_ (e.g., pCount, p_count)



Declaration Pitfall: A Common Mistake

- The * in a declaration applies **only to the variable immediately following it**.
- **Incorrect Assumption:**

```
// This does NOT declare two pointers!  
int *countPtr, count;
```

- countPtr is a pointer to an int (int *).
 - count is just a regular int
- **Best Practice:** Declare each pointer on its own line to avoid confusion

```
int *countPtr;  
int count;
```


Pointer Initialization

- **Crucial:** Always initialize pointers to prevent them from pointing to a random, invalid memory location.
- **Three Ways to Initialize:**
 - **NULL:** A symbolic constant (in <stddef.h>) that means the pointer points to nothing. **This is the preferred method.**

```
int *ptr = NULL;
```
 - **0:** Equivalent to NULL. The value 0 is the only integer that can be directly assigned to a pointer

```
int *ptr = 0;
```
 - **An Address:** Assigning the memory address of another variable (covered in the next section).

The Address Operator: &

- **Purpose:** Gets the **memory address** of its operand.
- **Syntax:** &variableName
- **How it Works:**
 - Given a variable, & returns where it's located in memory.
 - This address can then be assigned to a pointer.
- **Rule:** The operand of & **must be a variable**. It cannot be a literal value (e.g., &5) or an expression.

```
int y = 5;  
// Initialize yPtr with the memory address of y  
int *yPtr = &y;  
  
// We now say that "yPtr points to y"
```

The Indirection Operator: *

- **Purpose:** Accesses the **value** stored at the address a pointer is holding.
- **Also called:** The **Dereferencing Operator**.
- **Syntax:** *pointerName
- **How it Works:**
 - It follows the pointer to its stored memory address and retrieves the data from that location.

```
// yPtr holds the address of y.  
// *yPtr accesses the value at that address.  
printf("%d", *yPtr); // This will print 5
```


CRITICAL WARNING: Dereferencing Uninitialized Pointers

- Accessing the value of a pointer that doesn't point to a valid memory location is a **major error**.
- **Never dereference a pointer that has not been initialized!**
- **Potential Consequences:**
 - **Fatal Crash:** Your program may stop working immediately.
 - **Data Corruption:** It could accidentally modify important data, leading to incorrect results that are hard to debug.
 - **Security Breaches:** Can create vulnerabilities that could be exploited.

& and * are Complements

- The address (&) and indirection (*) operators are inverses of each other.
- Given `int y = 5;` and `int *yPtr = &y;`
- Applying * to a pointer **gets the original value.**
 - `*yPtr` is equivalent to `y` (both are 5).
- Applying & to a dereferenced pointer **gets the original address.**
 - `&(*yPtr)` is equivalent to `&y`.
- They effectively “cancel each other out.”

Practical Tip: Printing Addresses

- To see the memory addresses that pointers store, use the %p conversion specifier in printf.
- It displays the memory address in a system-specific format (usually hexadecimal).

```
int a = 7;  
int *aPtr = &a;  
  
printf("The address of a is %p\n", &a);  
printf("The value of aPtr is %p\n", aPtr);
```

The address of a is 0x7ffc1a3b4c54
The value of aPtr is 0x7ffc1a3b4c54

```

1 // fig07_01.c
2 // Using the & and * pointer operators.
3 #include <stdio.h>
4
5 int main(void) {
6
7     int a = 7;
8     int *aPtr = &a; // set aPtr to the address of a
9
10    printf("Address of a is %p\nValue of aPtr is %p\n\n", &a, aPtr);
11    printf("Value of a is %d\nValue of *aPtr is %d\n\n", a, *aPtr);
12    printf("Showing that * and & are complements of each other\n");
13    printf("&*aPtr = %p\n*&aPtr = %p\n", &*aPtr, *&aPtr);
14 }

```

```

Address of a is 0x7fffe69386cc
Value of aPtr is 0x7fffe69386cc

Value of a is 7
Value of *aPtr is 7

Showing that * and & are complements of each other
&*aPtr = 0x7fffe69386cc
*&aPtr = 0x7fffe69386cc

```

Key Takeaways

- **& (Address Operator):** Gets the **address** of a variable.
 - `&y` → “Where does `y` live in memory?”
- *** (Indirection Operator):** Gets the **value** at an address.
 - `*yPtr` → “What is the value at the location `yPtr` points to?”
- **Golden Rule: ALWAYS** initialize a pointer before you dereference (*) it.
- The `%p` format specifier is used to print pointer addresses.

Operators	Grouping	Type
() [] ++ (<i>postfix</i>) -- (<i>postfix</i>)	left to right	postfix
+ - ++ -- ! * & (<i>type</i>)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Passing Arguments to Functions in C

- **Pass-by-Value (The Default)**

- The function receives a **copy** of the argument's value.
- The original variable in the calling function is **unaffected** and cannot be changed.
- Safe and prevents accidental side effects.

- **Pass-by-Reference**

- The function receives the **memory address** of the argument.
- The function can directly access and **modify** the original variable.
- Achieved using pointers (& and *).

How Pass-by-Value Works

- A temporary copy of the variable is created inside the function.
- All operations are performed on this local copy.
- When the function ends, the copy is destroyed.
- **The original variable remains unchanged.**

```
void cubeByValue(int n) {  
    n = n * n * n; // Modifies the local copy 'n'  
}
```

```
// In main():  
int number = 5;  
cubeByValue(number);  
// 'number' is STILL 5 here.
```


How Pass-by-Reference Works

- This is a 3-step process using pointer operators:
 - **Caller:** Pass the **address** of the variable using the & operator.
 - **Function Definition:** The parameter must be a **pointer** (using *) to receive the address.
 - **Inside Function:** Use the **dereference operator (*)** to access and modify the original value.

```
void cubeByReference(int *nPtr) {  
    *nPtr = (*nPtr) * (*nPtr) * (*nPtr); // Modifies original value  
}
```

```
// In main():
```

```
int number = 5;
```

```
cubeByReference(&number); // Step 1: Pass the address
```

```
// 'number' is now 125!
```

Why Use Pass-by-Reference?

- It provides capabilities that pass-by-value cannot:
-  **Modify Caller's Data:** The primary reason. Allows a function to make lasting changes to variables outside its own scope.
-  **“Return” Multiple Values:** A function can only have one return statement, but by accepting multiple pointers, it can modify several variables in the caller.
-  **Efficiency:** Avoids the performance overhead of copying large data structures (like structs). Passing a small address is much faster than copying a large object.
-

Special Case: Arrays

- In C, arrays are **always passed by reference** by default.
- The name of an array is essentially a pointer to its first element (&arrayName[0]).
- You **do not** use the & operator when passing an array.
- **Compiler Interchangeability:** In a function's parameter list, the compiler treats `int arr[]` and `int *arr` as identical.

// These two function prototypes are seen as the same by the compiler:

```
void processArray(int arr[], int size);
```

```
void processArray(int *arr, int size);
```

Summary and Best Practices

Feature	Pass-by-Value	Pass-by-Reference
What is Passed?	A copy of the value	The memory address of the variable
Can Original Be Modified?	No	Yes
How to Call	myFunc(variable);	myFunc(&variable);
Function Parameter	void myFunc(int n);	void myFunc(int *nPtr);
Use When...	You only need to use the value.	You need to change the original variable.

Principle of Least Privilege: Use **pass-by-value** as your default. Only use pass-by-reference when the function is specifically required to modify the caller's data. This makes your code safer and easier to understand

Why Use the const Qualifier?

- `const` tells the compiler that a value should not be modified.
- **Enforces Least Privilege:** A function gets only the permissions it needs, and no more.
- **Reduces Bugs:** The compiler catches accidental modifications at compile-time, preventing runtime errors and unintentional side effects.
- **Improves Maintainability:** Makes code easier to understand and safer to modify.
- **A Key to Modernizing Legacy Code:** Many older C programs can be improved by correctly adding `const`.

The Four Combinations of const and Pointers

- There are four ways to use const with a pointer, each granting different permissions.
- **Non-Constant Pointer to Non-Constant Data** (Most permissions)
 - `int *ptr;`
- **Non-Constant Pointer to Constant Data**
 - `const int *ptr;`
- **Constant Pointer to Non-Constant Data**
 - `int * const ptr;`
- **Constant Pointer to Constant Data** (Least permissions)
 - `const int * const ptr;`

Type 1: Non-Constant Pointer to Non-Constant Data

- `char *sPtr;`
- **What it means:** The highest level of access.
- **Can the pointer be changed to point elsewhere?**  Yes. (`sPtr++`)
- **Can the data it points to be modified?**  Yes. (`*sPtr = 'A'`)
- **Use Case:** When a function needs to both modify the data and iterate through an array by moving the pointer.
 - *Example from text: A function `convertToUppercase` that changes characters in place.*

```

1 // fig07_06.c
2 // Converting a string to uppercase using a
3 // non-constant pointer to non-constant data.
4 #include <ctype.h>
5 #include <stdio.h>
6
7 void convertToUppercase(char *sPtr); // prototype
8
9 int main(void) {
10     char string[] = "cHaRaCters and $32.98"; // initialize char array
11
12     printf("The string before conversion is: %s\n", string);
13     convertToUppercase(string);
14     printf("The string after conversion is: %s\n", string);
15 }
16
17 // convert string to uppercase letters
18 void convertToUppercase(char *sPtr) {
19     while (*sPtr != '\0') { // current character is not
20         *sPtr = toupper(*sPtr); // convert to uppercase
21         ++sPtr; // make sPtr point to the next character
22     }
23 }

```

The string before conversion is: cHaRaCters and \$32.98
The string after conversion is: CHARACTERS AND \$32.98

Type 2: Non-Constant Pointer to Constant Data

- `const char *sPtr;` (Read right-to-left: “sPtr is a pointer to a char that is const”)
- **What it means:** The pointer can move, but the data it points to is read-only.
- **Can the pointer be changed to point elsewhere?**  Yes. (`sPtr++`)
- **Can the data it points to be modified?**  No. (`*sPtr = 'A' -> COMPILER ERROR`)
- **Use Case:** Safely reading data (like printing a string) without any risk of changing it.
 - *This is also used to pass large structs efficiently (by reference) while getting the safety of pass-by-value.*

```

1 // fig07_07.c
2 // Printing a string one character at a time using
3 // a non-constant pointer to constant data.
4
5 #include <stdio.h>
6
7 void printCharacters(const char *sPtr);
8
9 int main(void) {
10     // initialize char array
11     char string[] = "print characters of a string";
12
13     puts("The string is:");
14     printCharacters(string);
15     puts("");
16 }
17
18 // sPtr cannot be used to modify the character to which it points,
19 // i.e., sPtr is a "read-only" pointer
20 void printCharacters(const char *sPtr) {
21     // loop through entire string
22     for (; *sPtr != ; ++sPtr) { // no initialization
23         printf("%c", *sPtr);
24     }
25 }

```

The string is:
print characters of a string

```

1 // fig07_08.c
2 // Attempting to modify data through a
3 // non-constant pointer to constant data.
4 #include <stdio.h>
5 void f(const int *xPtr); // prototype
6
7 int main(void) {
8     int y = 7; // define y
9
10    f(&y); // f attempts illegal modification
11 }
12
13 // xPtr cannot be used to modify the
14 // value of the variable to which it points
15 void f(const int *xPtr) {
16     *xPtr = 100; // error: cannot modify a const object
17 }

```

fig07_08.c(16,5): error C2166: l-value specifies const object

Fig. 7.8 | Attempting to modify data through a non-constant pointer to constant data.

Type 3: Constant Pointer to Non-Constant Data

- `int * const ptr;` (Read right-to-left: “ptr is a const pointer to an int”)
- **What it means:** The pointer is locked to one memory address, but the data at that location can be changed.
- **Can the pointer be changed to point elsewhere?** ❌ **No.** (`ptr = &y` -> **COMPILER ERROR**)
- **Can the data it points to be modified?** ✅ **Yes.** (`*ptr = 100`)
- **Use Case:** When a pointer must always refer to the same variable, like an array name. Must be initialized when declared.

```

1 // fig07_09.c
2 // Attempting to modify a constant pointer to non-constant data.
3 #include <stdio.h>
4
5 int main(void) {
6     int x = 0; // define x
7     int y = 0; // define y
8
9     // ptr is a constant pointer to an integer that can be modified
10    // through ptr, but ptr always points to the same memory location
11    int * const ptr = &x;
12
13    *ptr = 7; // allowed: *ptr is not const
14    ptr = &y; // error: ptr is const; cannot assign new address
15 }

```

fig07_09.c(14,4): error C2166: l-value specifies const object

Fig. 7.9 | Attempting to modify a constant pointer to non-constant data.

Type 4: Constant Pointer to Constant Data

- `const int * const ptr;` (Read right-to-left: “ptr is a const pointer to an int that is const”)
- **What it means:** The most restrictive. A read-only pointer that points to a read-only location.
- **Can the pointer be changed to point elsewhere?** ❌ **No.** (`ptr = &y -> COMPILER ERROR`)
- **Can the data it points to be modified?** ❌ **No.** (`*ptr = 100 -> COMPILER ERROR`)
- **Use Case:** For maximum safety, when a function should only read a value from a fixed memory location (e.g., a hardware register or a fixed lookup table).

```

4
5 int main(void) {
6     int x = 5;
7     int y = 0;
8
9     // ptr is a constant pointer to a constant integer. ptr always
10    // points to the same location; the integer at that location
11    // cannot be modified
12    const int *const ptr = &x; // initialization is OK
13
14    printf("%d\n", *ptr);
15    *ptr = 7; // error: *ptr is const; cannot assign new value
16    ptr = &y; // error: ptr is const; cannot assign new address
17 }

```

```

fig07_10.c(15,5): error C2166: l-value specifies const object
fig07_10.c(16,4): error C2166: l-value specifies const object

```

Fig. 7.10 | Attempting to modify a constant pointer to constant data. (Part 2 of 2.)

Summary & Guiding Principle

- **Let the Principle of Least Privilege be your guide.**
- Always give a function *just enough* access to do its job, and absolutely no more.

If a function needs to...	Use this parameter type:	Pointer Movable?	Data Modifiable?
Modify an array's contents	<code>int * ptr</code>	✓	✓
Read an array's contents without changing them	<code>const int * ptr</code>	✓	✗
Modify a single, fixed variable via a pointer	<code>int * const ptr</code>	✗	✓
Read a single, fixed variable via a pointer	<code>const int * const ptr</code>	✗	✗

Bubble Sort with Pass-by-Reference

- The objective is to refactor the Bubble Sort algorithm into two distinct functions:
- **bubbleSort(array, size)**
 - Contains the main sorting logic (the loops).
- **swap(pointer1, pointer2)**
 - A helper function responsible for only one thing: exchanging two values.
- This separation of concerns makes the code cleaner and more modular.

The Challenge: Modifying Array Elements

- **Problem:** C enforces **information hiding**. The swap function does not have direct access to the array variable inside bubbleSort.
- Individual array elements (e.g., array[j]) are simple values (scalars). By default, they are **passed-by-value**.
- A copy of array[j] would be sent to swap, and swapping copies would have **no effect** on the original array.
- **How do we allow swap to modify the actual array elements?**

The Solution: Pass-by-Reference for Elements

- We use pointers to give swap access to the original data.
- **In bubbleSort:** We pass the **memory address** of each element using the & operator.

```
// Pass the addresses of the two elements to be swapped  
swap(&array[j], &array[j + 1]);
```

- **In swap:** The function parameters are defined as **pointers** to receive these addresses.

```
void swap(int *element1Ptr, int *element2Ptr) { ... }
```

- Inside swap, *element1Ptr now becomes a synonym for the original array[j].

Inside the swap Function Logic

- By dereferencing the pointers, swap can now modify the values in the original array.
- Even though swap cannot access array[j] by name...
- ...it achieves the exact same effect by using the pointers it received.

```
// This code modifies the original array elements in bubbleSort  
void swap(int *element1Ptr, int *element2Ptr) {  
    int hold = *element1Ptr;  
    *element1Ptr = *element2Ptr;  
    *element2Ptr = hold;  
}
```

Software Engineering Principle 1: Reusability

- **Why must we pass the array size to bubbleSort?**
- When you pass an array to a function, you are only passing the memory address of its first element. The address itself contains **no information about the array's length**.
- **Passing the size makes the function reusable.** bubbleSort can now work on an integer array of *any* size.
- **Alternatives are Bad Practice:**
 - **Hard-coding the size:** Makes the function useless for any other array size.
 - **Using a global variable:** Violates the principle of least privilege and prevents the function from being easily used in other programs.

Software Engineering Principle 2: Least Privilege

- **Why is the prototype for swap placed inside bubbleSort's body?**

```
void bubbleSort(int * const array, const size_t size) {  
    void swap(int *element1Ptr, int *element2Ptr); // Prototype is here!  
    // ... loops and calls to swap() ...  
}
```

- This is a design choice that enforces the **Principle of Least Privilege**.
- By placing the prototype locally, swap is only “visible” and callable from within bubbleSort.
- This prevents other functions in the program from accidentally (or intentionally) calling a helper function that they are not supposed to use.

```

1 // fig07_11.c
2 // Putting values into an array, sorting the values into
3 // ascending order and printing the resulting array.
4 #include <stdio.h>
5 #define SIZE 10
6
7 void bubbleSort(int * const array, size_t size); // prototype
8
9 int main(void) {
10     // initialize array a
11     int a[SIZE] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
12
13     puts("Data items in original order");
14
15     // loop through array a
16     for (size_t i = 0; i < SIZE; ++i) {
17         printf("%4d", a[i]);
18     }
19
20     bubbleSort(a, SIZE); // sort the array
21
22     puts("\nData items in ascending order");
23
24     // loop through array a
25     for (size_t i = 0; i < SIZE; ++i) {
26         printf("%4d", a[i]);
27     }
28
29     puts("");
30 }

```

```

32 // sort an array of integers using bubble sort algorithm
33 void bubbleSort(int * const array, size_t size) {
34     void swap(int *element1Ptr, int *element2Ptr); // prototype
35
36     // loop to control passes
37     for (int pass = 0; pass < size - 1; ++pass) {
38         // loop to control comparisons during each pass
39         for (size_t j = 0; j < size - 1; ++j) {
40             // swap adjacent elements if they're out of order
41             if (array[j] > array[j + 1]) {
42                 swap(&array[j], &array[j + 1]);
43             }
44         }
45     }
46 }

```



```

48 // swap values at memory locations to which element1Ptr and
49 // element2Ptr point
50 void swap(int *element1Ptr, int *element2Ptr) {
51     int hold = *element1Ptr;
52     *element1Ptr = *element2Ptr;
53     *element2Ptr = hold;
54 }

```

Data items in original order									
2	6	4	8	10	12	89	68	45	37
Data items in ascending order									
2	4	6	8	10	12	37	45	68	89

Fig. 7.11 | Putting values into an array, sorting the values into ascending order and printing the resulting array. (Part 2 of 2.)

What is the sizeof Operator?

- A **unary operator** that determines the size of its operand in **bytes**.
- The result is a `size_t` value.
- It's a **compile-time** operator, meaning it's resolved during compilation and has no performance cost when the program runs.
 - *(Exception: Variable-Length Arrays, VLAs)*
- **Syntax:**
 - With a type: `sizeof(int)` (Parentheses are **required**)
 - With a variable: `sizeof array` or `sizeof(array)` (Parentheses are optional)

sizeof with Arrays

- This is a primary and powerful use of the operator.
- When applied to an array name, sizeof returns the **total number of bytes occupied by the entire array**.

```
// A float is typically 4 bytes  
float prices[20];  
  
// This calculates the total size: 20 elements * 4 bytes/element  
size_t total_bytes = sizeof(prices);  
  
printf("Total size of array is: %zu bytes\n", total_bytes);
```

Total size of **array** is: 80 bytes

CRITICAL PITFALL: sizeof with Pointers

- This is the most common point of confusion when using sizeof.
 - When an array is passed to a function, what the function actually receives is a **pointer** to the array's first element.
 - Applying sizeof to that pointer **returns the size of the pointer variable itself**, NOT the size of the original array.

```
// This function will NOT return the array's size  
void getSize(int *ptr) {  
    printf("Size inside function: %zu\n", sizeof(ptr));  
}
```

Size inside function: 8 // On a 64-bit system where pointers are 8 bytes

```
int main() {  
    int numbers[20]; // sizeof(numbers) here is 20 * 4 = 80  
    getSize(numbers);  
}
```

The “Number of Elements” Idiom

- You can use `sizeof` to calculate the number of elements in an array.
 - **The Formula:** `sizeof(arrayName) / sizeof(arrayName[0])`
 - **How it Works:**
 - It divides the total bytes of the array by the number of bytes in a single element.

```
double measurements[22]; // A double is typically 8 bytes
```

```
// (22 * 8) / 8 = 176 / 8 = 22
```

```
size_t count = sizeof(measurements) / sizeof(measurements[0]);
```

```
printf("The array has %zu elements.\n", count);
```

The **array** has 22 elements.

Portability and General Usage

- **Problem:** The size of data types (int, long, pointers, etc.) can vary across different systems and compilers (e.g., 32-bit vs. 64-bit machines).
- **Solution:** Use sizeof to write **portable code** that doesn't depend on hard-coded, assumed sizes. This makes your programs more robust and less likely to fail when compiled on a different machine.

- ```
// This code will always show the correct sizes for the system it's compiled on
printf("Size of int: %zu bytes\n", sizeof(int));
printf("Size of double: %zu bytes\n", sizeof(double));
printf("Size of a pointer: %zu bytes\n", sizeof(int*));
```



# Key Takeaways

- sizeof returns the size of a variable or type in **bytes**.
- sizeof(array) -> Gives the **total size** of the array.
- sizeof(pointer) -> Gives the size of the **pointer variable** itself (typically 4 or 8 bytes).
- To find the number of elements in an array, use: `sizeof(array) / sizeof(array[0])`
- Rely on sizeof to make your code portable across different computer systems.

# Valid Pointer Arithmetic Operations

- Not all standard math operators are valid with pointers. Pointer arithmetic is specifically designed for navigating arrays.
- The allowed operations are:
  -  **Incrementing (++)** and **Decrementing (--)** a pointer.
  -  **Adding an integer** to a pointer (+ or +=).
  -  **Subtracting an integer** from a pointer (- or -=).
  -  **Subtracting two pointers** from each other (if they point to the same array).
-  **Rule:** These operations are only meaningful when the pointers point to elements within the same array.

# The Core Concept: Scaled by Type Size

- Pointer arithmetic is **not** conventional arithmetic.
- When you add an integer N to a pointer, the address is increased by:  $N * \text{sizeof}(\text{the\_type\_the\_pointer\_points\_to})$
- **Example:** Assume int is 4 bytes.

```
int v[5]; // Array of integers
int *vPtr = v; // vPtr points to v[0] at address 3000

vPtr += 2; // This does NOT result in 3002!
```

- **The Calculation:**  $3000 + (2 * \text{sizeof}(\text{int})) \rightarrow 3000 + (2 * 4) \rightarrow 3008$
- The pointer vPtr now correctly points to the element v[2].



# Pointer Operations in Practice

- Given `int *vPtr` pointing to `v[0]` (address 3000):
- **Incrementing (++):** `vPtr++;`
  - Moves the pointer to the *next element* (`v[1]` at address 3004).
- **Decrementing (--):** `vPtr--;`
  - Moves the pointer to the *previous element*.
- **Adding an Integer:** `vPtr = vPtr + 3;`
  - Moves the pointer forward 3 elements (`v[3]` at address 3012).
- **Subtracting an Integer:** `vPtr = vPtr - 1;`
  - Moves the pointer back 1 element

# Subtracting Two Pointers

- This operation reveals the distance between two elements.
- When you subtract two pointers (that point to the same array), the result is **not the difference in bytes**, but the **number of elements** between them.

```
int v[5];
int *vPtr1 = &v[0]; // Address is 3000
int *vPtr2 = &v[2]; // Address is 3008

int x = vPtr2 - vPtr1;
```

- The value of x will be **2**, because there are two int elements between the pointers.
- The raw byte difference ( $3008 - 3000 = 8$ ) is automatically scaled down by `sizeof(int)`.

# Relationship Between Pointers and Arrays

- **Array Name as Pointer**
- **Array name is a constant pointer:**

```
int b[5];
```

```
int *bPtr;
```

```
bPtr = b; // Same as: bPtr = &b[0];
```

**Cannot modify array name:**

- `b += 3;`      *// INVALID - compilation error*

# Four Ways to Access Array Elements

- *// 1. Array subscript notation*
- `b[3]`
- *// 2. Pointer/offset with array name*
- `*(b + 3)`
- *// 3. Pointer subscripting; refers to the array element b[3].*
- `bPtr[3]`
- *// 4. Pointer/offset with pointer*
- `*(bPtr + 3)`
- *All four reference the same memory location*

```

6 int main(void) {
7 int b[] = {10, 20, 30, 40}; // create and initialize array b
8 int *bPtr = b; // create bPtr and point it to array b
9
10 // output array b using array subscript notation
11 puts("Array b printed with:\nArray subscript notation");
12
13 // loop through array b
14 for (size_t i = 0; i < ARRAY_SIZE; ++i) {
15 printf("b[%zu] = %d\n", i, b[i]);
16 }
17
18 // output array b using array name and pointer/offset notation
19 puts("\nPointer/offset notation where the pointer is the array name");
20
21 // loop through array b
22 for (size_t offset = 0; offset < ARRAY_SIZE; ++offset) {
23 printf("*(b + %zu) = %d\n", offset, *(b + offset));
24 }
25
26 // output array b using bPtr and array subscript notation
27 puts("\nPointer subscript notation");
28
29 // loop through array b
30 for (size_t i = 0; i < ARRAY_SIZE; ++i) {
31 printf("bPtr[%zu] = %d\n", i, bPtr[i]);
32 }
33
34 // output array b using bPtr and pointer/offset notation
35 puts("\nPointer/offset notation");
36
37 // loop through array b
38 for (size_t offset = 0; offset < ARRAY_SIZE; ++offset) {
39 printf("*(bPtr + %zu) = %d\n", offset, *(bPtr + offset));
40 }
41 }

```

Array b printed with:  
 Array subscript notation  
 b[0] = 10  
 b[1] = 20  
 b[2] = 30  
 b[3] = 40

Pointer/offset notation where the pointer is the array name  
 \*(b + 0) = 10  
 \*(b + 1) = 20  
 \*(b + 2) = 30  
 \*(b + 3) = 40

**Fig. 7.14** | Using subscripting and pointer notations with arrays. (Part 2 of 3.)

Pointer subscript notation  
 bPtr[0] = 10  
 bPtr[1] = 20  
 bPtr[2] = 30  
 bPtr[3] = 40

Pointer/offset notation  
 \*(bPtr + 0) = 10  
 \*(bPtr + 1) = 20  
 \*(bPtr + 2) = 30  
 \*(bPtr + 3) = 40



# String Copying Examples

- **Array subscript notation:**

```
void copy1(char *s1, const char *s2) {
 for (int i = 0; (s1[i] = s2[i]) != '\0'; i++);}
```

- **Pointer arithmetic:**

```
void copy2(char *s1, const char *s2) {
 for (; (*s1 = *s2) != '\0'; s1++, s2++);}
```

- **Both accomplish the same task!**

```

1 // fig07_15.c
2 // Copying a string using array notation and pointer notation.
3 #include <stdio.h>
4 #define SIZE 10
5
6 void copy1(char * const s1, const char * const s2); // prototype
7 void copy2(char *s1, const char *s2); // prototype
8
9 int main(void) {
10 char string1[SIZE]; // create array string1
11 char *string2 = "Hello"; // create a pointer to a string
12
13 copy1(string1, string2);
14 printf("string1 = %s\n", string1);
15
16 char string3[SIZE]; // create array string3
17 char string4[] = "Good Bye"; // create an array containing a string
18
19 copy2(string3, string4);
20 printf("string3 = %s\n", string3);
21 }
22
23 // copy s2 to s1 using array notation
24 void copy1(char * const s1, const char * const s2) {
25 // loop through strings
26 for (size_t i = 0; (s1[i] = s2[i]) != ; ++i) {
27 ; // do nothing in body
28 }
29 }

```

```

30
31 // copy s2 to s1 using pointer notation
32 void copy2(char *s1, const char *s2) {
33 // loop through strings
34 for (; (*s1 = *s2) != ; ++s1, ++s2) {
35 ; // do nothing in body
36 }
37 }

```

```

string1 = Hello
string3 = Good Bye

```



# Arrays of Pointers

- **Arrays of Pointers: String Arrays in C**

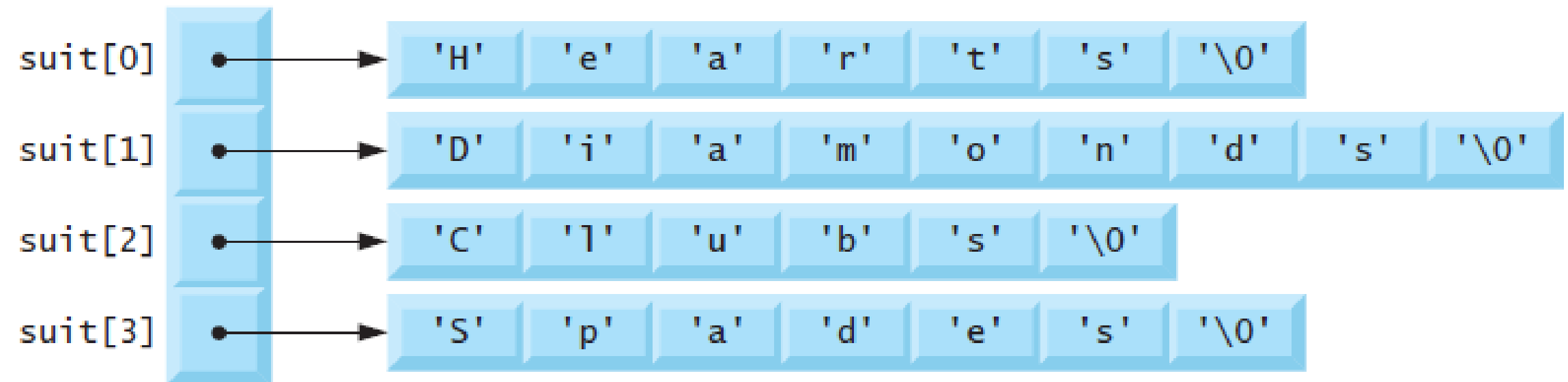
- **Key Concept:**

Arrays can contain pointers as elements, commonly used to create arrays of strings where each element is a pointer to a string's first character.



# String Array Example

```
const char *suit[4] = {
 "Hearts",
 "Diamonds",
 "Clubs",
 "Spades"
};
```



## What this means:

- `suit[4]` → Array of 4 elements
- `char *` → Each element is a pointer to char
- `const` → Strings cannot be modified
- Each pointer points to the first character of its string

```

#include <stdio.h>

int main() {
 // Array of pointers to strings
 const char *fruits[] = {
 "Apple", // Each element is a pointer to string literal
 "Banana", // Stored in read-only memory
 "Cherry", // Different length strings
 "Durian",
 "Elderberry" // Longest string (9 chars + null terminator)
 };

 int size = sizeof(fruits) / sizeof(fruits[0]);

 printf("=== Basic String Array ===\n");
 for (int i = 0; i < size; i++) {
 printf("fruits[%d] = %s (stored at address: %p)\n",
 i, fruits[i], (void*)fruits[i]);
 }

 return 0;
}

```

=== Basic String Array ===

fruits[0] = Apple (stored at address: 0x104b6bf34)

fruits[1] = Banana (stored at address: 0x104b6bf3a)

fruits[2] = Cherry (stored at address: 0x104b6bf41)

fruits[3] = Durian (stored at address: 0x104b6bf47)

fruits[4] = Elderberry (stored at address: 0x104b6bf4e)

# Function Pointers in C

- **Key Concept:**

A function pointer contains the address in memory where the function's code begins. Like array names, function names represent memory addresses.

- **Similar to Arrays:**

- Array name → Address of first element
- Function name → Address of function code

# Function Pointer Declarations

*// Function that returns int pointer*

```
int *func(int a, int b);
```

*// Pointer to function returning int*

```
int (*funcPtr)(int a, int b);
```

- **Crucial Difference:**
- Parentheses group \* with pointer name
- Without parentheses: function returning pointer
- With parentheses: pointer to function

```

1 // fig07_17.c
2 // Multipurpose sorting program using function pointers.
3 #include <stdio.h>
4 #define SIZE 10
5
6 // prototypes
7 void bubbleSort(int work[], size_t size, int (*compare)(int a, int b));
8 int ascending(int a, int b);
9 int descending(int a, int b);
10
46 // multipurpose bubble sort; parameter compare is a pointer to
47 // the comparison function that determines sorting order
48 void bubbleSort(int work[], size_t size, int (*compare)(int a, int b)) {
49 void swap(int *element1Ptr, int *element2ptr); // prototype
50
51 // loop to control passes
52 for (int pass = 1; pass < size; ++pass) {
53 // loop to control number of comparisons per pass
54 for (size_t count = 0; count < size - 1; ++count) {
55 // if adjacent elements are out of order, swap them
56 if ((*compare)(work[count], work[count + 1])) {
57 swap(&work[count], &work[count + 1]);
58 }
59 }
60 }
61 }
62

```



```
63 // swap values at memory locations to which element1Ptr and
64 // element2Ptr point
65 void swap(int *element1Ptr, int *element2Ptr) {
66 int hold = *element1Ptr;
67 *element1Ptr = *element2Ptr;
68 *element2Ptr = hold;
69 }
70
71 // determine whether elements are out of order for an ascending order sort
72 int ascending(int a, int b) {
73 return b < a; // should swap if b is less than a
74 }
75
76 // determine whether elements are out of order for a descending order sort
77 int descending(int a, int b) {
78 return b > a; // should swap if b is greater than a
79 }
```

```

11 int main(void) {
12 // initialize unordered array a
13 int a[SIZE] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
14
15 printf("%s", "Enter 1 to sort in ascending order,\n"
16 "Enter 2 to sort in descending order: ");
17 int order = 0;
18 scanf("%d", &order);
19
20 puts("\nData items in original order");
21
22 // output original array
23 for (size_t counter = 0; counter < SIZE; ++counter) {
24 printf("%5d", a[counter]);
25 }
26
27 // sort array in ascending order; pass function ascending as an
28 // argument to specify ascending sorting order
29 if (order == 1) {
30 bubbleSort(a, SIZE, ascending);
31 puts("\nData items in ascending order");
32 }
33 else { // pass function descending
34 bubbleSort(a, SIZE, descending);
35 puts("\nData items in descending order");
36 }
37
38 // output sorted array
39 for (size_t counter = 0; counter < SIZE; ++counter) {
40 printf("%5d", a[counter]);
41 }
42
43 puts("\n");
44 }
45

```

Enter 1 to sort in ascending order,  
Enter 2 to sort in descending order: 1

Data items in original order

2 6 4 8 10 12 89 68 45 37

Data items in ascending order

2 4 6 8 10 12 37 45 68 89

Enter 1 to sort in ascending order,  
Enter 2 to sort in descending order: 2

Data items in original order

2 6 4 8 10 12 89 68 45 37

Data items in descending order

89 68 45 37 12 10 8 6 4 2

# Array of Function Pointers

*// Array of 3 function pointers*

```
void (*f[3])(int) = {function1, function2, function3};
```

- **Reading the Declaration:**
- f is an array [3]
- \*f of pointers to functions
- (int) that take an int parameter
- void and return void
- **Initialization with function names**
- **Example: Using Function Pointers to Create a Menu-Driven System**



```

1 // fig07_18.c
2 // Demonstrating an array of pointers to functions.
3 #include <stdio.h>
4
5 // prototypes
6 void function1(int a);
7 void function2(int b);
8 void function3(int c);
9
10 int main(void) {
11 // initialize array of 3 pointers to functions that each take an
12 // int argument and return void
13 void (*f[3])(int) = {function1, function2, function3};
14
15 printf("%s", "Enter a number between 0 and 2, 3 to end: ");
16 int choice = 0;
17 scanf("%d", &choice);
18
19 // process user
20 while (choice >= 0 && choice < 3) {
21 // invoke function at location choice in array f and pass
22 // choice as an argument
23 (*f[choice])(choice);
24
25 printf("%s", "Enter a number between 0 and 2, 3 to end: ");
26 scanf("%d", &choice);
27 }
28
29 puts("Program execution completed.");
30 }
31

```

```

32 void function1(int a) {
33 printf("You entered %d so function1 was called\n\n", a);
34 }
35
36 void function2(int b) {
37 printf("You entered %d so function2 was called\n\n", b);
38 }
39
40 void function3(int c) {
41 printf("You entered %d so function3 was called\n\n", c);
42 }

```

Enter a number between 0 and 2, 3 to end: 0  
You entered 0 so function1 was called

Enter a number between 0 and 2, 3 to end: 1  
You entered 1 so function2 was called

Enter a number between 0 and 2, 3 to end: 2  
You entered 2 so function3 was called

Enter a number between 0 and 2, 3 to end: 3  
Program execution completed.

# The Generic Pointer: void \*

- **What it is:** A void \* is a generic pointer that can hold the address of any data type.
- **Assignment:** You can assign any pointer type to a void \* (and vice-versa) without an explicit cast.

```
int x = 10;
void *genericPtr = &x; // Perfectly valid
```

- **CRITICAL LIMITATION:** You cannot dereference a void \*.
- \*genericPtr = 20; // COMPILER ERROR
- **Why?** The compiler doesn't know the type (and therefore the size) of the data it points to. It doesn't know if it should modify 1, 4, or 8 bytes of memory.

# Benefits:

- Clean, extensible design
- Easy to add new menu options
- No complex switch/if-else chains
- Function table approach